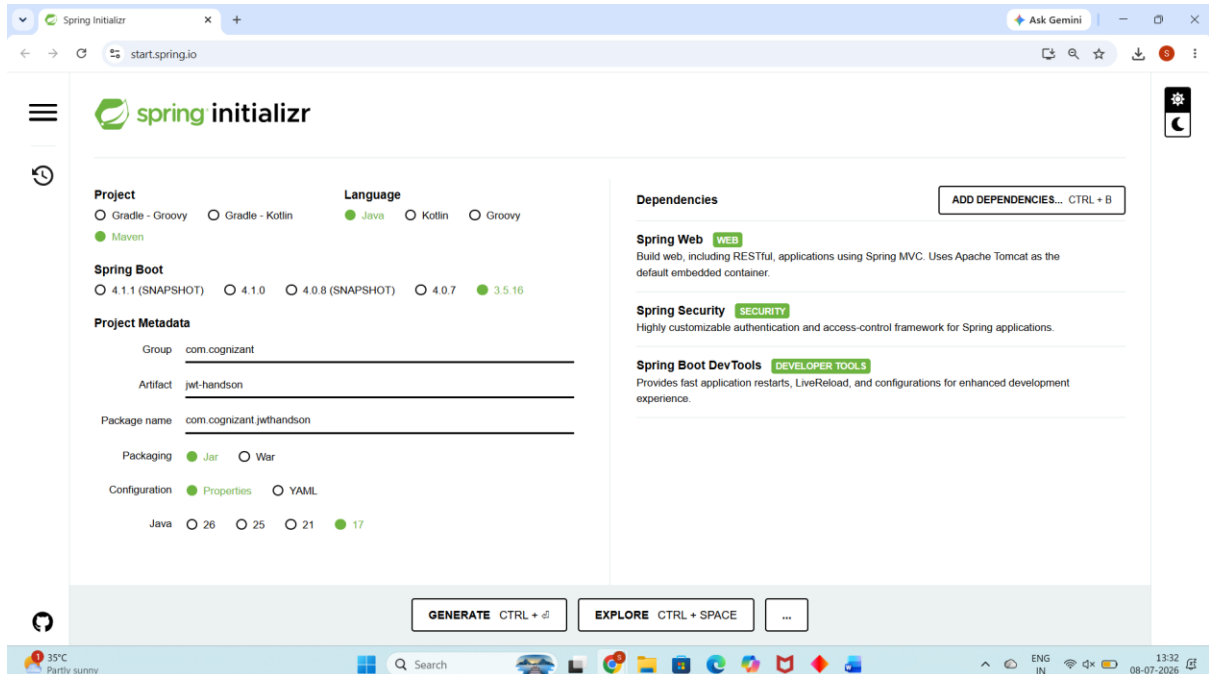


COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

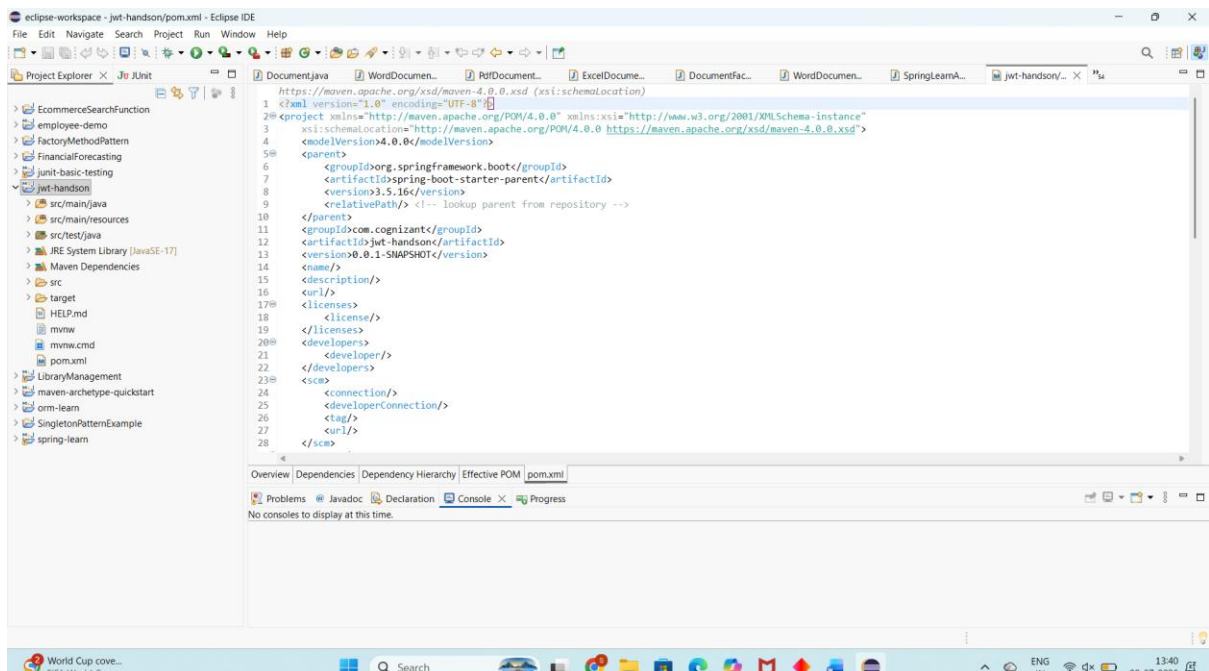
Spring REST using Spring Boot 3

Create authentication service that returns JWT

Step 1 : Creating JWT Authentication Project Using Spring Initializr

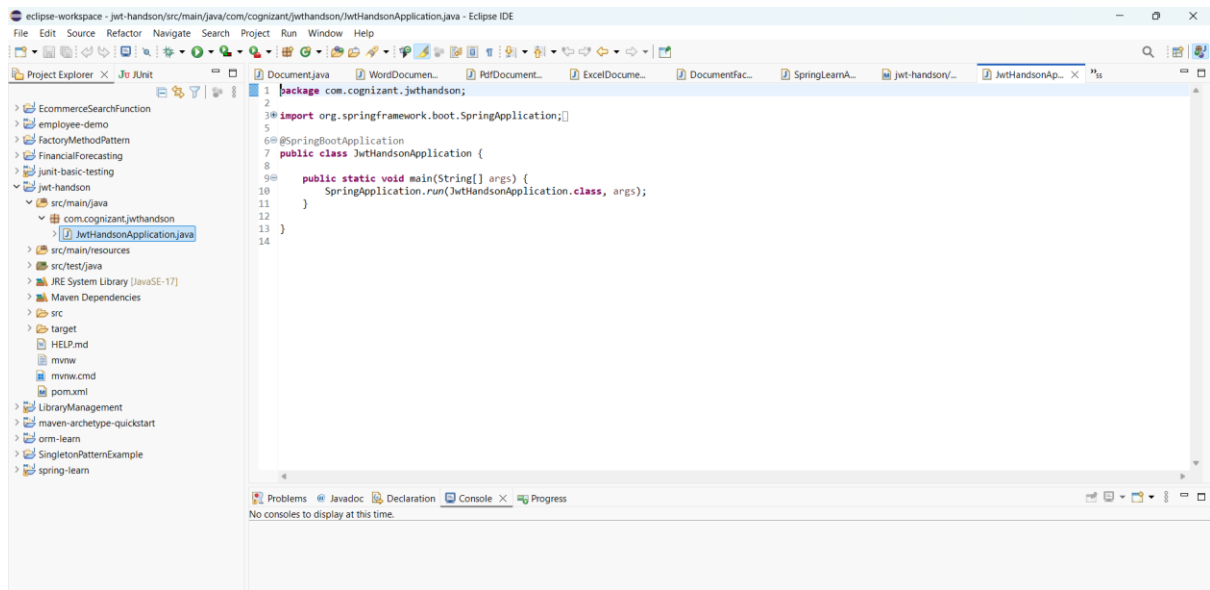


Step 2 : Spring Boot Project Configuration in pom.xml

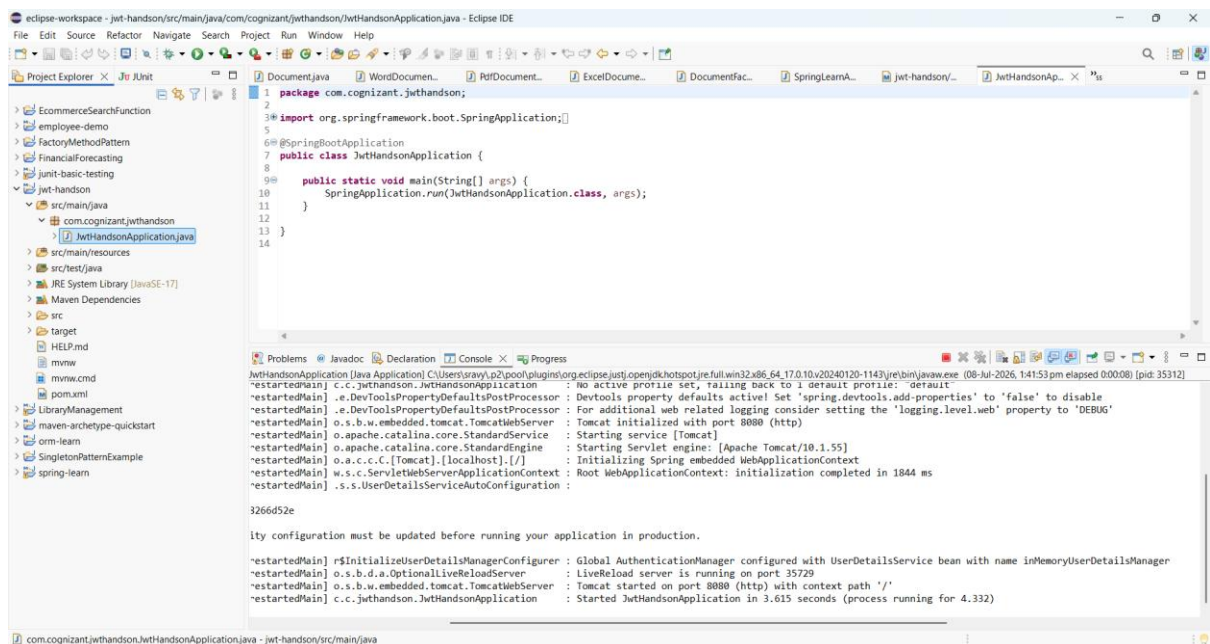


COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 3 : JWT Hands-On Spring Boot Main Application Class

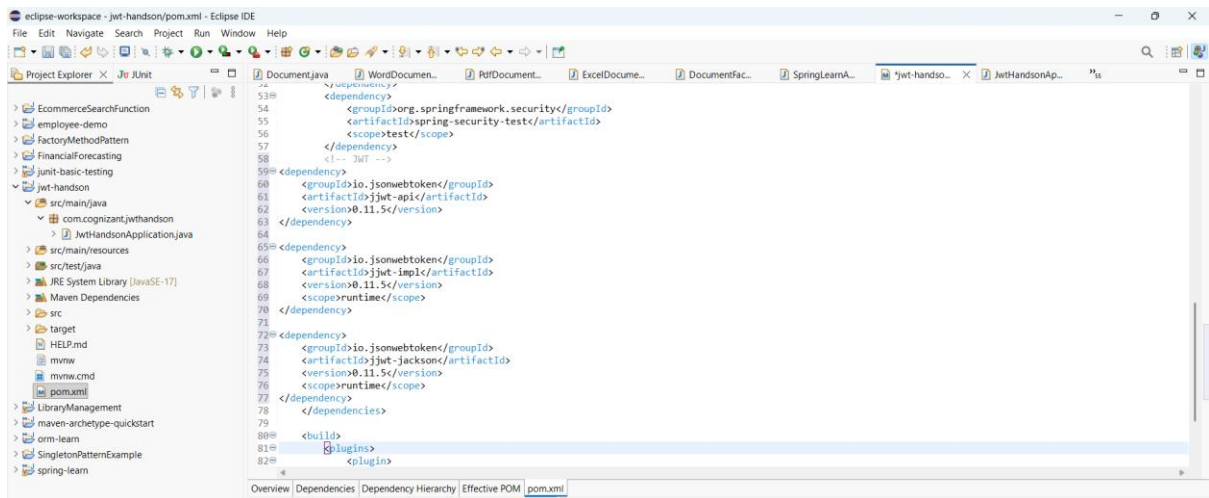


Step 4 : Running the JWT Authentication Spring Boot Application

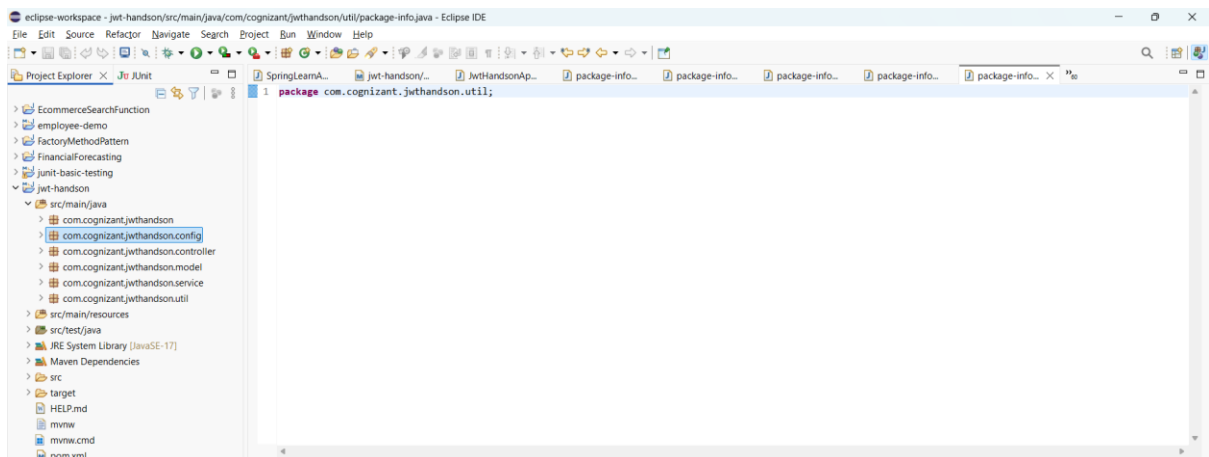


COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

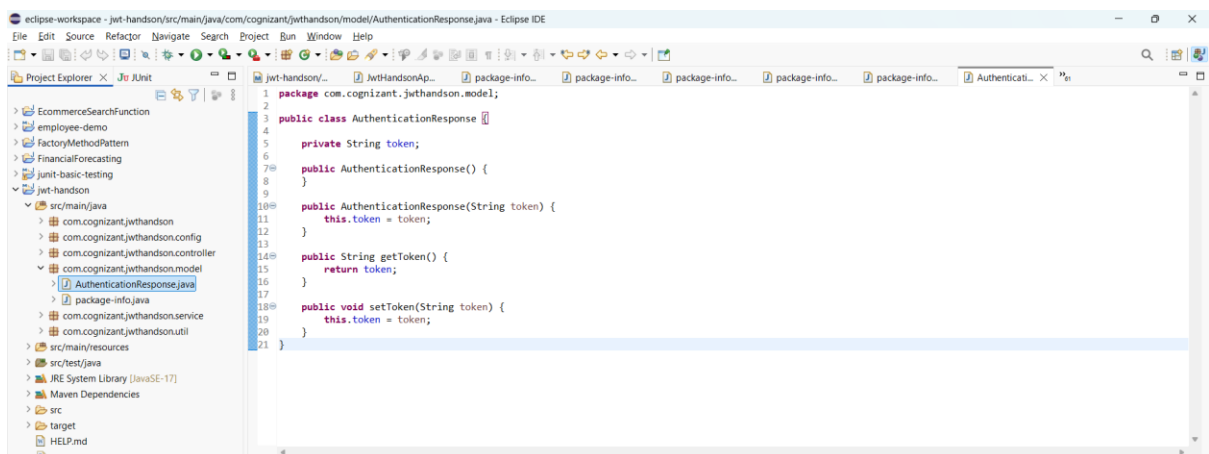
Step 5 : Adding JWT Dependencies in pom.xml



Step 6 : Creating Package Structure for JWT Authentication

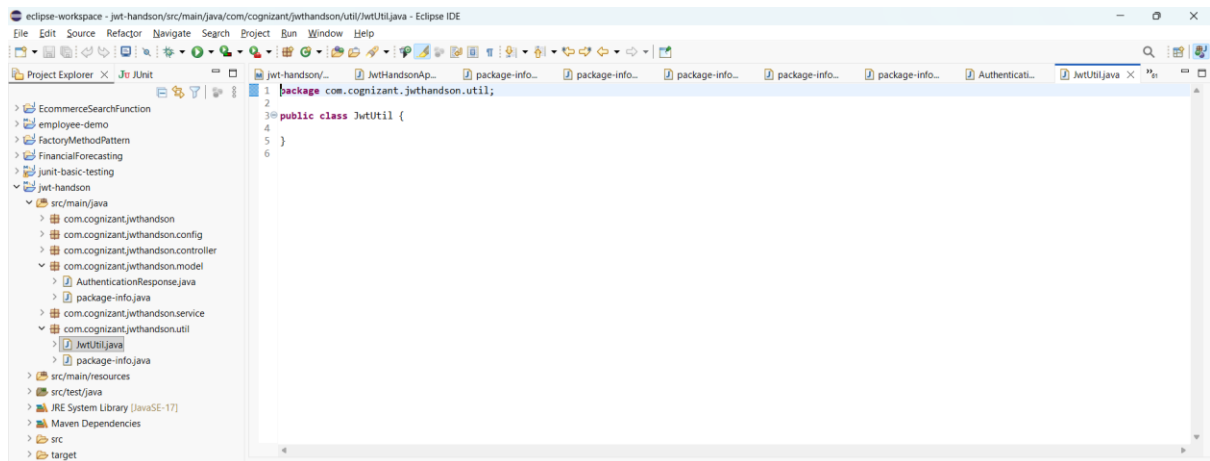


Step 7 : Creating Authentication Response Model Class

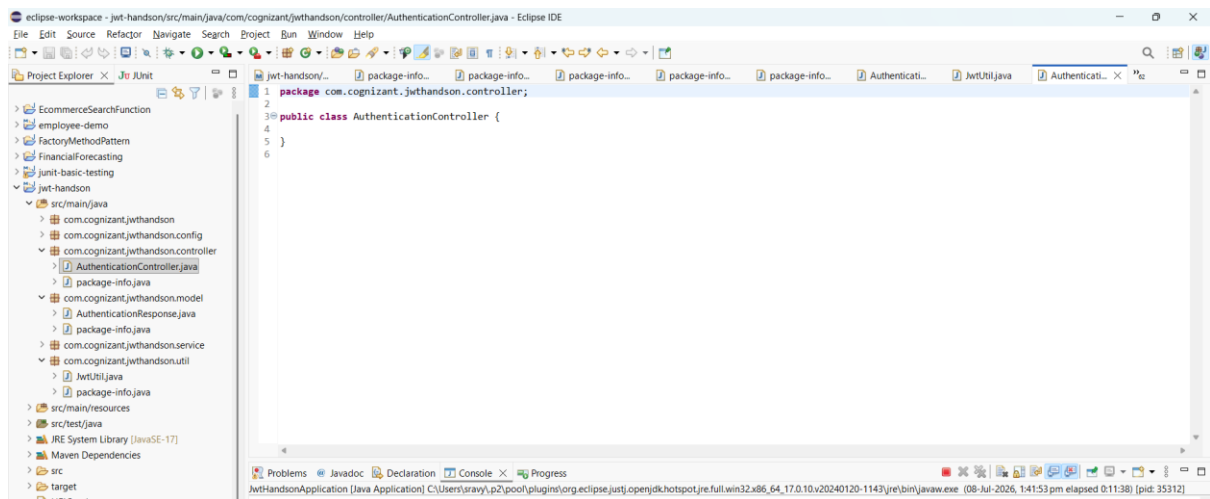


COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

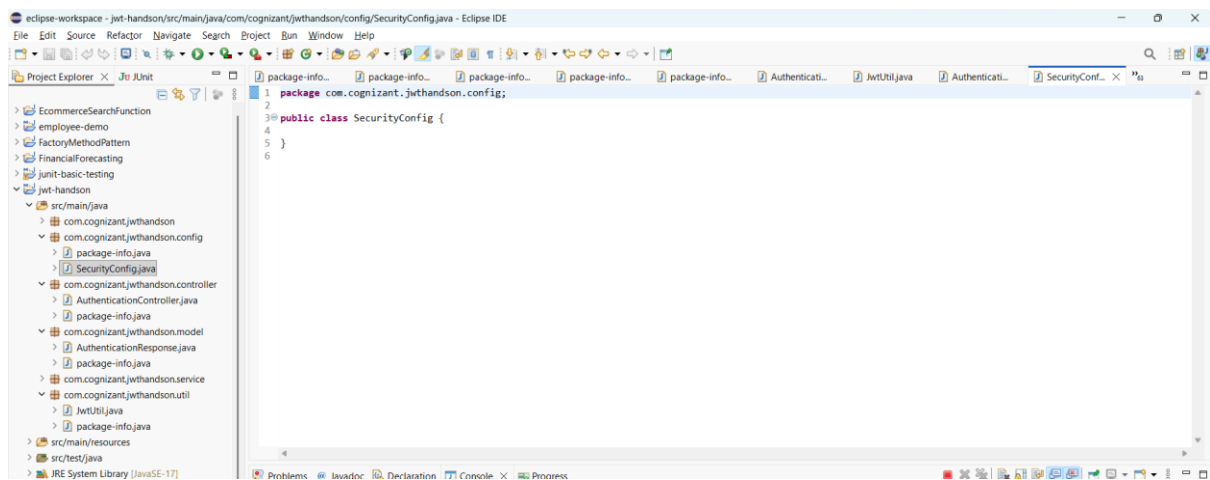
Step 8 : Creating JWT Utility Class



Step 9 : Creating Authentication Controller

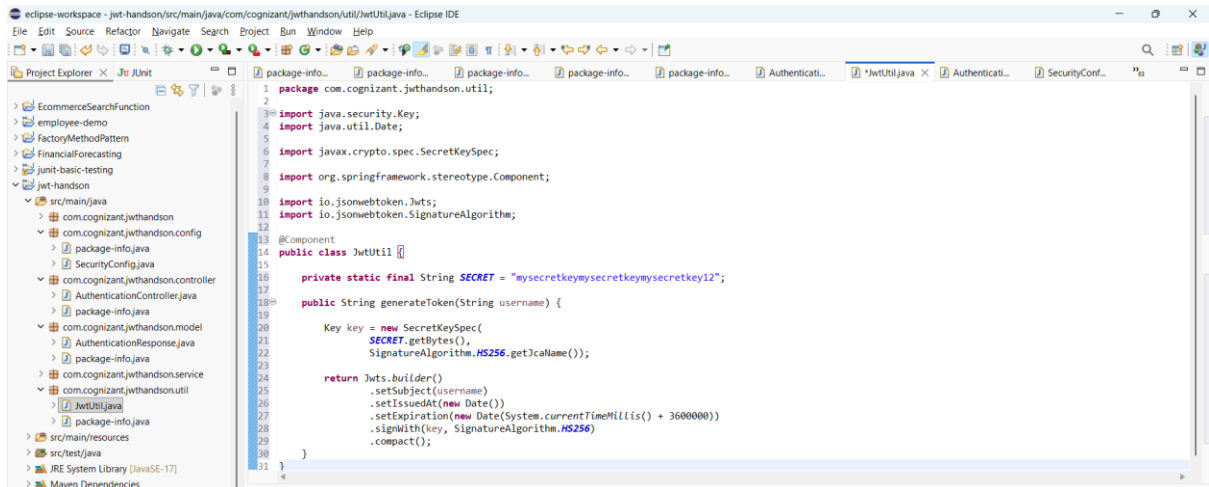


Step 10 : Creating Spring Security Configuration



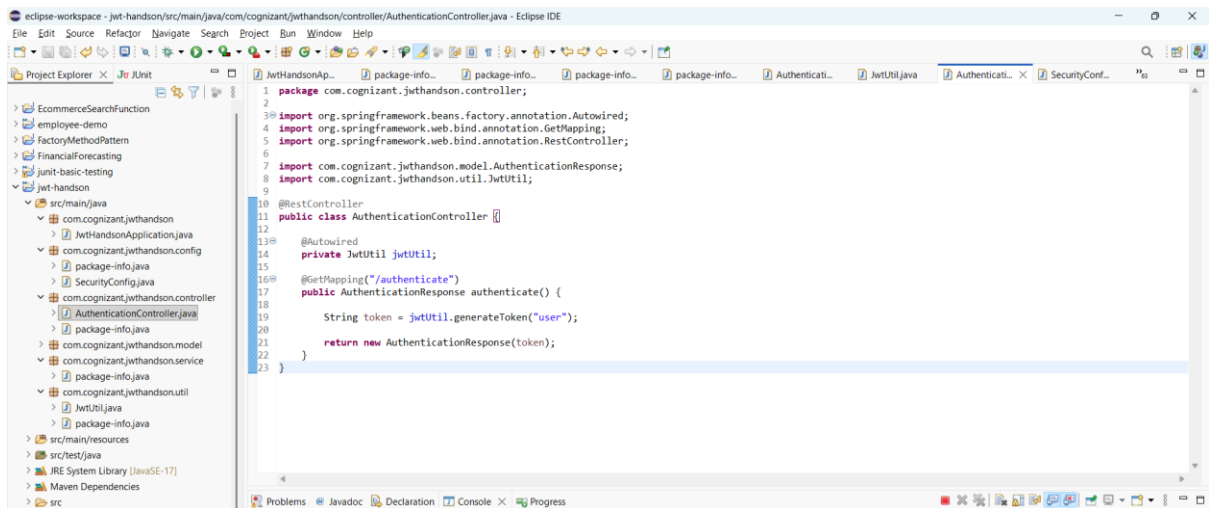
COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 11 : Implementing JWT Token Generation Logic



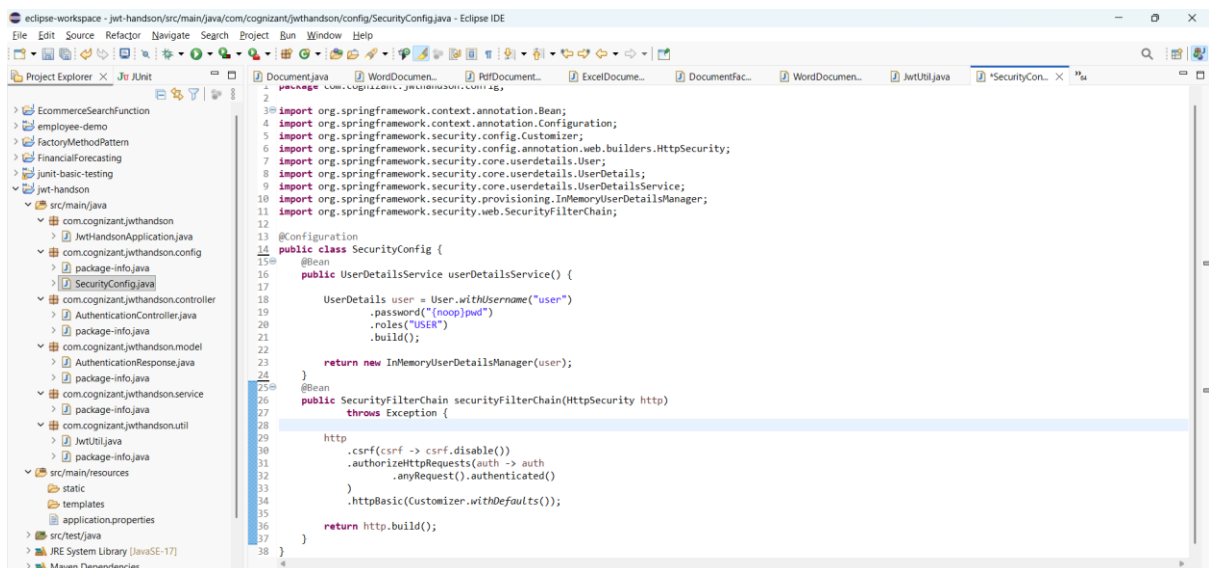
```
1 package com.cognizant.jwthandson.util;
2
3 import java.security.Key;
4 import java.util.Date;
5
6 import javax.crypto.spec.SecretKeySpec;
7
8 import org.springframework.stereotype.Component;
9
10 import io.jsonwebtoken.Jwts;
11 import io.jsonwebtoken.SignatureAlgorithm;
12
13 @Component
14 public class JwUtil {
15
16     private static final String SECRET = "mysecretkeymysecretkeymysecretkey12";
17
18     public String generateToken(String username) {
19
20         Key key = new SecretKeySpec(
21             SECRET.getBytes(),
22             SignatureAlgorithm.HS256.getName());
23
24         return Jwts.builder()
25             .setSubject(username)
26             .setIssuedAt(new Date())
27             .setExpiration(new Date(System.currentTimeMillis() + 3600000))
28             .signWith(key, SignatureAlgorithm.HS256)
29             .compact();
30     }
31 }
```

Step 12 : Implementing Authentication Endpoint



```
1 package com.cognizant.jwthandson.controller;
2
3 import org.springframework.beans.factory.annotation.Autowired;
4 import org.springframework.web.bind.annotation.GetMapping;
5 import org.springframework.web.bind.annotation.RestController;
6
7 import com.cognizant.jwthandson.model.AuthenticationResponse;
8 import com.cognizant.jwthandson.util.JwUtil;
9
10 @RestController
11 public class AuthenticationController {
12
13     @Autowired
14     private JwUtil jwUtil;
15
16     @GetMapping("/authenticate")
17     public AuthenticationResponse authenticate() {
18
19         String token = jwUtil.generateToken("user");
20
21         return new AuthenticationResponse(token);
22     }
23 }
```

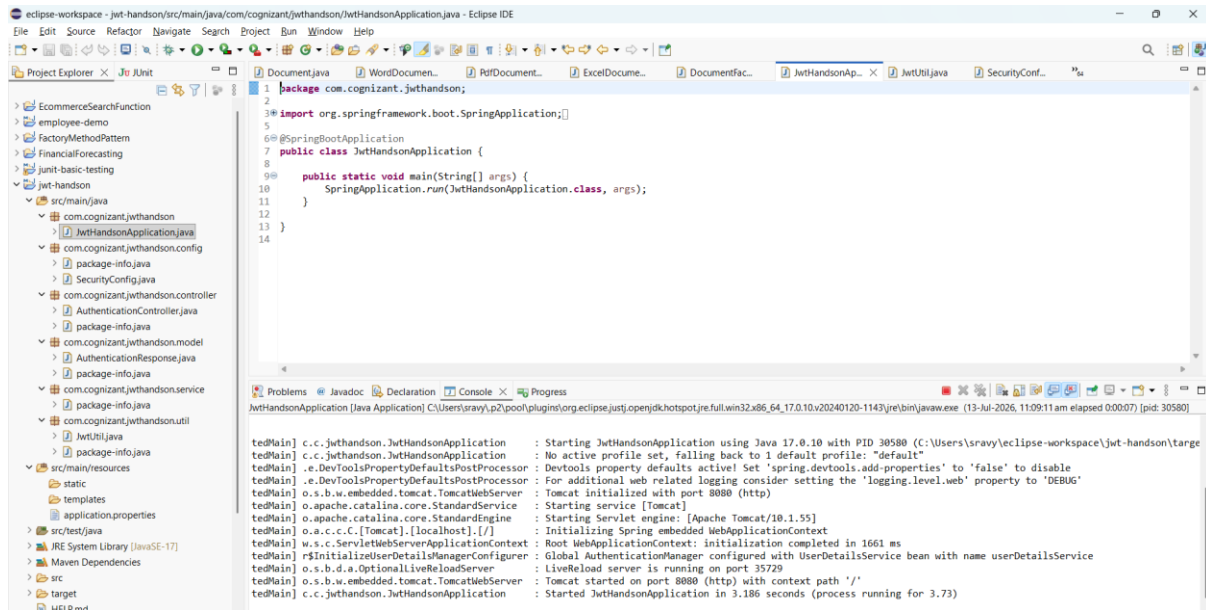
Step 13 : Configuring User Authentication and Security Filter Chain



```
1 package com.cognizant.jwthandson.config;
2
3 import org.springframework.context.annotation.Bean;
4 import org.springframework.context.annotation.Configuration;
5 import org.springframework.security.config.annotation.web.builders.HttpSecurity;
6 import org.springframework.security.config.annotation.web.configuration.EnableWebSecurity;
7 import org.springframework.security.core.userdetails.User;
8 import org.springframework.security.core.userdetails.UserDetails;
9 import org.springframework.security.core.userdetails.UserDetailsService;
10 import org.springframework.security.provisioning.InMemoryUserDetailsManager;
11 import org.springframework.security.web.SecurityFilterChain;
12
13 @Configuration
14 public class SecurityConfig {
15
16     @Bean
17     public UserDetailsService userDetailsService() {
18
19         UserDetails user = User.withUsername("user")
20             .password("{noop}pwd")
21             .roles("USER")
22             .build();
23
24         return new InMemoryUserDetailsManager(user);
25     }
26
27     @Bean
28     public SecurityFilterChain securityFilterChain(HttpSecurity http)
29         throws Exception {
30
31         http
32             .csrf(csrf -> csrf.disable())
33             .authorizeHttpRequests(auth -> auth
34                 .anyRequest().authenticated())
35             .httpBasic(Customizer.withDefaults());
36
37         return http.build();
38     }
39 }
```

COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

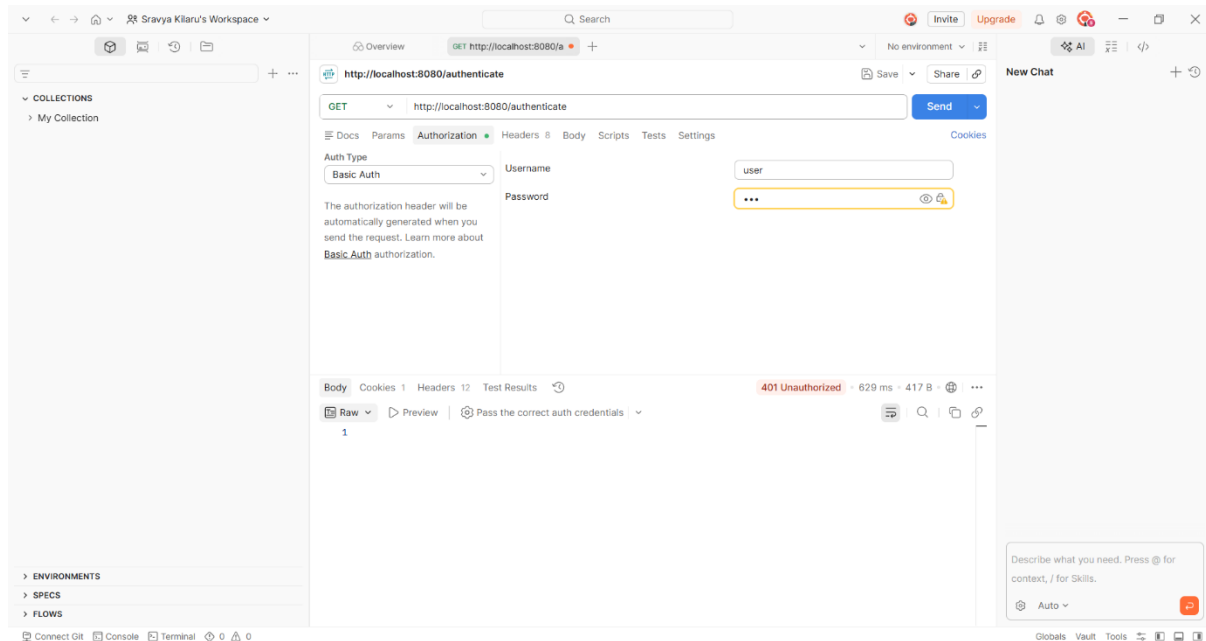
Step 14 : Successful Execution of JWT Authentication Application



```
1 package com.cognizant.jwtHandson;
2
3 import org.springframework.boot.SpringApplication;
4
5 @SpringBootApplication
6 public class JwtHandsonApplication {
7
8     public static void main(String[] args) {
9         SpringApplication.run(JwtHandsonApplication.class, args);
10    }
11 }
12
13
14
```

```
tedMain] c.c.jwtHandson.JwtHandsonApplication : Starting JwtHandsonApplication using Java 17.0.10 with PID 30580 (C:\Users\srvay\workspace\jwt-handson\targe
tedMain] c.c.jwtHandson.JwtHandsonApplication : No active profile set, falling back to 1 default profile: "default"
tedMain] .e.DevToolsPropertyDefaultsPostProcessor : Devtools property defaults active! Set 'spring.devtools.add-properties' to 'false' to disable
tedMain] .e.DevToolsPropertyDefaultsPostProcessor : For additional web related logging consider setting the 'logging.level.web' property to 'DEBUG'
tedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initialized with port 8080 (http)
tedMain] o.apache.catalina.core.StandardWebServer : Starting service [Tomcat]
tedMain] o.apache.catalina.core.StandardEngine : Starting Servlet engine: [Apache Tomcat/10.1.55]
tedMain] o.a.c.c.C.[Tomcat].[localhost].[/] : Initializing Spring embedded WebApplicationContext
tedMain] w.s.c.ServletWebServerApplicationContext : Root WebApplicationContext: initialization completed in 1661 ms
tedMain] r.InitializeUserDetailsServiceConfigurer : Global AuthenticationManager configured with UserDetailsService bean with name userDetailsService
tedMain] o.s.b.d.a.OptionalLiveReloadServer : LiveReload server is running on port 35729
tedMain] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat started on port 8080 (http) with context path '/'
tedMain] c.c.jwtHandson.JwtHandsonApplication : Started JwtHandsonApplication in 3.186 seconds (process running for 3.73)
```

Step 15 : Testing Authentication with Invalid Credentials



COGNIZANT DIGITAL NURTURE 5.0 – JAVA FSE

Step 16 : Successful Authentication and JWT Token Generation

